* Handling the Exceptions in python:-

→ Handling Exception is nothing converting Technical error messages into into user friendly error messages.

→ TO Handle the exceptions in python, we have 5 keywords they are:-

1. try
2. except
3. else
4. Finally
5. raise.

Syntax for Handling the exception in python.

```
try:
    Block of statements
    Generating exceptions
except exception-class-name-1:
        Block of statements
        Generates user-friendly error message.
except exception-class-name-2:
        Block of statements
        Generates use friendly error message.
- - -     - - -  - - -  - - -
- - -     - - -  - - -  - - -
except exception-class-name-n:
        Block of statements
        Generates user friendly error message.
else:
    Block of statements
    generates Result.
finally:
    Block of statements
    compulsorily
```

* Explanation for the Keywords used in syntax of Handling exception:-

① try

→ it is the block in which we write block of statements generating exceptions. in otherwords what are all the statements generating exceptions, those statement must be written try block and Hence try block is called exception monitoring Block.

→ when an exception occurs in try block then prm comes out of try block and executes appropriate except block.

→ After executing appropriate except block, PVM never goes to try block for executing rest of the statements in try block.

→ Every try block must be immediately followed by except block (otherwise we get syntax error).

→ Every try blocks must contain atleast one except Block. it is recommended to write multiple except blocks for generating user-friendly error messages.

②. except

→ it is the block in which we write block of statements generates user-friendly error-messages. in otherwords except block suppreses Technical error messages and generates user-friendly error message and Hence except block is called Exception processing Block

Note: Handling Exception = try block + except block.

→ except Block will execute when there is an exception occurs in try block.

→ Even we write multiple except blocks, PVM executes appropriate except block (single Block) depends on type of ~~execution~~ exception occurs in try block.

→ The place for writing except block is that after try block and before else block.

③ **else:**

→ it is the block in which we write block of statements will display results of the program and hence else block is called "result Generated Block"

→ else block will execute when there is no exception occurs in try Block.

→ writing else block is optional.

→ The place of writing else block is that after except block and before finally block (if it present)

④ **finally:**

→ it is the block, in which we write block of statements will relinquish (release/ close/ given-up/ clean-up) the resources (files, Databases softwares) which are obtained in try block and finally block is called resources relinquishing Block.

→ finally block will execute compulsorily

→ finally block is optional to write.

→ The place of writing finally block is that after else block (if else block present)

---

program for accepting two numerical values from KBD and find their DIV.

ex①.

```
s1 = input("Enter first value:")
s2 = input("Enter second value:")
a = int(s1)   # exception generated statement
                problemtatic statement
b = int(s2)   #   ''          ''
c = a/b
print("Div = {}".format(c))
```

ex②

```
try:
    print("I am in try block")
    Same likhna Hai.
except ZeroDivisionError:
        print("Don't enter zero for Deno...")
except ValueError:
        print("Don't Enter Alnums, sprs, symbols")
else:
    print("------")
    print("Div = {}".format(c))
    print("-  -  -")
finally:
    print("I am from finally Block")
```

ex③.

```
except (ZeroDivisionError, ValueError)   # multi exceptions
    Baaki as it is.                        Handling Block
```

ex④

```
except ZeroDivisionError as z:
        print(z)
except ValueError as v:
        print(v)
```

try:
1. open a file/DB connection (resource/Gathering)
2
3
4
5. c = 100/10
:
100. Close the file/release the DB connection
except —
except —
else: —
finally: relinquishing (close/release/giveup/cleanup)
→ 100.

ex⑤

except:
    print("oops .... something went wrong")

Baaki sab same ex④ ke raisa.

ex⑥

#DivEX6.PY ---- 2023-May KVR developed this program with 2 exceptions.
#2025 --- there is chance of modifying the code by adding some new statement by new programmer.

try:
    print("I am in try block")
    s1 = input("Enter first value")
    s2 = input("Enter second value")
    a = int(s1)
    b = int(s2)   } #exception
    c = a/b

    #s = "PYTHON" # 2025 Adding
    # print(s[11])
except ZeroDivisionError:
        print("Don't enter zero for denominator")
except value error:
        print("Don't enter Alphabets, spaces and symbols")
except: # default except block - last
    print("oops... something went wrong")

(Baki sab ex2 की तरह)

Day 55

* various forms of except Blocks:-

=> we can we except block in various forms.

Form 1: with this form, we can Handle one exception at a time.

```
try:
    - - -
except exception class-name 1:
    - - -
except exception class - name 2:
    - - -
except exception class - name n:
    - - -
```

Form 2: with this forms we can Handle multiple specific exception with a single except block this type of facility is called multi exception Handling block.

```
try:
    - - -
except (exception-class 1, exception-class 2 ... exception - class-n):
    - - -
        Block of statements provides
        user friendly error messages
        for multiple specific exceptions
```

Form 3: with this form, we can Handle one exception at a time with Alias name for capturing Technical error messages occurring due exception occurence.

```
try:
    - - -
except exception-class-name 1 as alias name:
    print (alias name)
except exception-class-name 2 as alias name:
    print (alias name)
except exception - class - name n as alias name:
    print (alias name)
```

Form(4) Default except block matches with all types of exceptions

```
try:
    - - - - -
    - - - - -
except:
    - - - -
    print("oops something went wrong")
    - - - - -
```

=> Industry is not recommended to write single default except block bcoz it is giving common messages for all types of exceptions.

=> Hence industry is highly recommended to write default except block at end of all types of specific except blocks.

final syntax

```
try:
    - - - - - -
except exception class name1:
    - - - - - - -
except exception classnamen:
    - - - - - - -
except ~~exception~~ # default except block.
    - - - - - - -
else:
    - - - -
finally: - - - -
```

(OR)

```
try:
    - - - - -
except (exception-class-1, exception-class2 ... exception
                                              class-n):
        Block of statement provides
        user-friendly error messages
        for multiple specific Exceptions.
except # default except block
    - - - - -
else:
    - - - -
finally:
    - - -
```

* Development of programmer-Defined exceptions:-

→ These exceptions developed by Python Lang. programmer and available in python project and used by other team members of same project for dealing with common problems in project

→ Some of the common problems are:-

a) Attempting to enter invalid pin in ATM Application (Ex: pin error)

b) Attempting to withdraw to more amount than existing Balance (Ex: insufficienBal)

c) Attempting to enter invalid user name and password ...etc.
(Ex: LoginError).

→ TO develop programmer defined exception classes, we use the following steps

step ① Choose the programmer class name

step ② Every programmer-Defined class Name must inherit from either Exception or BaseException and Hence programmer-Defined class is called programmer-Defined exception sub class:-

step ③ The step ① and step ② on some File name with an extension .py and it acts as module name

example:-

```
class pinError(Exception): pass
class InsufficientFundError(BaseException): pass
class LoginError(Exception: pass
```

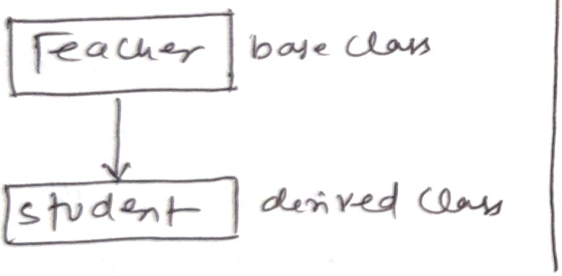
⇒ Here pinError, InsufficientFund error and Login error are called programmer-Defined exception.

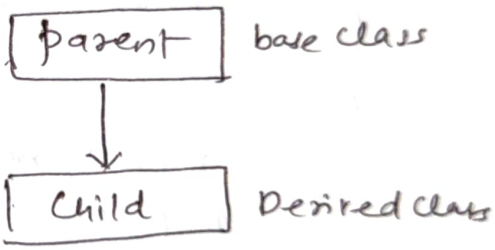
* Exception Handling Hierarchy Chart :-

Exception OR BaseException

- → ZeroDivisionError
- → KeyError
- → ValueError
- → TYPE
- → Index
- → modulo
- → unbound
- → Attribute
- → Name
- → syntax
- → indentation
- → FileNotFound
- → IOError
- → OS
- → File exist etc.

→ pre-defined exception subclasses





Class Teacher :
    pass

Class student (Teacher) : pass

Class parent :
    pass

Class Child (parent) : pass

→ Class ValueError (Exception) : pass

→ Class IndexError (BaseException) : pass

→ Class loginError (Exception) : pass
① ②

→ Class pinError (BaseException) : pass
① ②

* raise keyword :-

→ raise keyword is used for hitting / raising / generating the exception provided some condition must be satisfied.

→ raise keyword always used inside of function definition only.

→ PVM uses raise keyword implicitly for hitting pre-defined exceptions where as programmer makes the PVM to use raise keyword explicitly for hitting or Generating programmer-defined exceptions.

syntax -① if(Test Cond):
raise <exception-class-name>

syntax -② def functionname(list of formal params if any):
if (Test cond):
raise <exception-class-name>

Example).
```
from hyd import HydDivisionError
def division(a,b):
    if (b==0):
        raise HydDivisionError
    else:
        return(a/b)
```

Q. Write a python program which will develop an exception for Handling zero division error.

a) program for) Generating an exception when division by zero taking place.

b) program For) Handling the exception when zero is denominator.

→ file①
```
#Development of Exception
# Hyd.py ---  filename & modulename
class HydDivisionError(exception): pass
```

(2) # Generating Exception when division by zero by using raise kwd.
Division.py ---- file name & module name

```
from Hyd import HydDivisionError
def divop(a,b):
    if(b==0):
        raise HydDivisionError #raise keyword is used for hitting/generating the exception
    else:
        return(a/b)
```

(3) # Handling the exception by zero in denominator.
DivDemo.py

```
from Division import divop
from Hyd import HydDivisionError
a=int(input("Enter first value"))
b=int(input("Enter second value"))
try:
    res=divop(a,b) # function call with result or exception
except HydDivisionError:
    print("Don't Enter zero for denominator")
else:
    print("Div:{}".format(res))
```

4. Write a python program which will generate a multiplication table where n must be positive integer value
if n=0, then raise zero error.
if ~~negative~~ n<0 then raise negative number.

→

development of exception for generating multable
mulExceptions.py --- File name & module Name

```
class ZeroError(Exception): pass
class NegNumError(BaseException): pass
```

```
# program for multable by Hitting exceptions.
# multable.py  ---  file name or module
from MulExceptions import ZeroError, NegNumError
def multable(n):
    if (n==0):
        raise ZeroError    # Hitting the exception
    elif (n<0):
        raise NegNumError # Hitting the exception
    elif(n>0):
        print("-  -  -  -")
        print("multable for: {}".format(n))
        print("-  -  -  -  -")
        for i in range(1,11):
            print("\t{}x{} = {}".format(n,i,n*i))
        print("-  -  -  -  -")
```

→

```
# program for obtaining multable and Handling the exception
# multableDemo.py
from multable import multable.
from MulExceptions import ZeroError, NegNumError
try:
    n=int(input("Enter a numbr for Generating Mul Table:"))
    multable(n) # function call with result and exceptions.
except ZeroError:
    print("Don't enter zero For mul table:")
except NegNumError:
    print("Don't enter -ve number for mul table!")
except valueError:
    print("Don't enter Alnums, strs, and special symbols")
```

Day (58)

* ATM operation :-

1. Deposit
2. withdraw
3. Balenq
4. Exit

11. write a python program which will emplement sum of the ATM operation and developed test and Handle the exception if required.

Filesystem

1. ATMmenu.PY
2. ATMExceptions.PY
3. ATMoperations.PY
4. ATMmainprog.PY

```
# ATMmenu.PY
def menu():
    s="""\t\t ========== ATM operations ==========
    1. Deposit
    2. withDraw
    3. Balenq
    4. Exit
    ==========="""
    print(s).
```

```
# ATMExceptions.PY
class DepositError(Exception): pass
class withdrawError(BaseException): pass
class InsuffundError(Exception): pass
```

```
# ATMoperations.PY
from ATMExceptions import DepositError, withDrawError, insuffundError.
bal=500.00  # Global variables
def deposit():
    damt=float(input("Enter Ur Deposit Amount:"))  # pvm implicitly raises value error.
    if(damt<=0):
        raise DepositError
    else:
        global bal
        bal=bal+damt
```

```
        print("Ur Account xxxx123 credited with INR: {}".
                                        format(damt))
        print("Afts Deposit ur ACCount xxxx123 Bal INR:{}".
                                        format(bal))
def withdraw():
    global bal
    ~~bal=bal+damt~~
    ~~print~~ wamt = float(input("Ents ur withdraw Amount:"))
                                        #pvm implicitly
                                        raise ValueError.
    if(wamt<=0):
        raise withDrawError
    elif((wamt+500)>bal):
        raise insuffFundError
    else:
        bal = bal - wamt
        print("Ur Account xxxx123 Debited with INR:{}".format
                                        (wamt))
        print("Afts with Ur ACCount xxx123 Bal INR:{}".format
                                        (bal))

def baleng():
    print("\t Ur ACCount xxxxx123 Bal INR:{}".format(bal))
```

→

```
#ATM mainprog.py
from ATMmenu import menu
from ATMExceptions import DepositError, withdrawError, insuffFund
                                                    Error
from ATMoperations import deposit, withdraw, baleng
while(True):
    menu()
    ch= int(input("\t\t Ents Ur Choice:"))
    match(ch):
        case1:
            try:
                Deposit()
            except ValueError:
                print("Dont try Deposit Alnums, sps and special
                                        Symbols")
            except DepositError:
                Print("Dont try deposit zero/-ve values")
```

```
Case2:
    try:
      withdraw()
    except ValueError:
        print("Don't try withdraw Alnums, strs, and special symbols")
    except withdrawError:
        print("Don't try withdraw zero/-ve values")
    except InsuffFund error:
        print("Ur Account does not have suff-funds--
              read python note")
Case3:
    balenq()
Case4:
    print("Thx for this program")
    break
Case_:
    print("Ur selection of operation is wrong-Try Again")
```

→ Second approach me defrunatm(): ke andr main program ko daaliye

* pycharm me code ko run karne keliye:-

```
import sys
sys.path.append(fileAddress)
from ATMMainprog1 import runatm
runatm()
```

↙ other place pr run karne ke liye main program function defined krna pregg